

Decomposition in DBMS

In relational database design, a large relation (table) is often divided into smaller relations to remove redundancy and improve data integrity. This process is called **decomposition** and is commonly performed during **normalization**.

However, when a relation is decomposed, two important properties must be considered:

1. **Lossless (Non-loss) Join Property**
2. **Dependency Preserving Property**

An ideal decomposition satisfies **both properties**.

Lossless Decomposition (Lossless Join)

A decomposition of a relation R into relations $R_1, R_2, \dots, R_n$ is called lossless **if the original relation can be reconstructed exactly by performing a natural join of the decomposed relations**.

In simple terms:

If we split a table and later join the smaller tables, **we should obtain exactly the same original table without losing any information or generating spurious tuples**.

Mathematical Condition

For decomposition of R into R_1 and R_2 , the decomposition is lossless if:

$$R_1 \cap R_2 \rightarrow R_1$$

or

$$R_1 \cap R_2 \rightarrow R_2$$

This means the common attribute between the two relations must functionally determine at least one of the relations.

This means the common attribute between the two relations must functionally determine at least one of the relations.

Joining the decomposed tables gives back the original table.

No extra or missing rows are created.

To Check for Lossless Decomposition

For lossless decomposition, we have to ensure three major things. Suppose any relation R (A, B, C, D) is decomposed into relations R1 (A, B, C) and R2 (C, D), then check for :

- Union of attributes set of R1 & R2 should be equal to the attribute set of R, i.e., $R1 \cup R2 = R$.
- Intersection of attributes set of R1 & R2 should not be empty set, i.e., $R1 \cap R2 \neq \phi$
- Attribute closure of intersection of attributes set of R1 & R2 is either superset of R1 or R2, i.e., $(R1 \cap R2)^+ \supseteq R1$ or $(R1 \cap R2)^+ \supseteq R2$
- Not all decompositions into 1NF, 2NF, 3NF, or BCNF are guaranteed to be lossless.

Example of Lossless Decomposition

Consider relation:

STUDENT(RollNo, Name, Department)

RollNo	Name	Department
101	Rahul	CSE
102	Neha	ECE

Functional Dependency:

RollNo $\rightarrow$ Name, Department

Decomposition

Relation 1: R₁(RollNo, Name)

RollNo	Name
101	Rahul
102	Neha

Relation 2: $R_2(\text{RollNo}, \text{Department})$

RollNo	Department
101	CSE
102	ECE

Common attribute = **RollNo**

Since RollNo determines both Name and Department, joining the tables gives:

$R_1 \bowtie R_2$

RollNo	Name	Department
101	Rahul	CSE
102	Neha	ECE

The original relation is reconstructed perfectly.
Therefore, the decomposition is **lossless**.

Example 2

Consider the relation:

Employee (Employee_Id, Ename, Salary, Department_Id, Dname)

Decompose it using lossless decomposition into:

- **R1:** Employee_desc (Employee_Id, Ename, Salary, Department_Id)
- **R2:** Department_desc (Department_Id, Dname).

Here:

- Common attribute: Dept_Id
- Closure of Dept_Id (with proper FDs) should cover either R1 or R2 to ensure lossless join:

$(R1 \cap R2 \rightarrow R1) \text{ or } (R1 \cap R2 \rightarrow R2)$

Advantages of Lossless Decomposition

- Prevents loss of information
- Ensures correct reconstruction of the original relation
- Maintains data consistency
- Essential for database normalization

Lossy Decomposition

A decomposition is called lossy if the **original relation cannot be reconstructed exactly by joining the decomposed relations.**

In such cases, joining the relations may produce spurious tuples (extra incorrect records).

Example of Lossy Decomposition

Original relation:

R(Student, Course, Instructor)

Student	Course	Instructor
A	DBMS	X
B	OS	Y

Decomposition

R₁(Student, Course)

Student	Course
A	DBMS
B	OS

$R_2(\text{Course, Instructor})$

Course	Instructor
DBMS	X
OS	Y

If multiple students take the same course, joining these tables may create extra combinations of Student and Instructor that did not exist originally.

Thus the decomposition introduces incorrect tuples, making it lossy.

Problems with Lossy Decomposition

- Generates incorrect data combinations
- Original relation cannot be accurately reconstructed
- Leads to data inconsistency

Therefore, lossy decomposition should always be avoided.

Dependency Preserving Decomposition

A decomposition is called dependency preserving if all **functional dependencies of the original relation can still be enforced by the decomposed relations without performing a join.**

In other words:

Each functional dependency should be present in at least one of the decomposed relations.

Example of Dependency Preserving Decomposition

Relation:

$R(A, B, C)$

Functional Dependencies:

$A \rightarrow B$

$B \rightarrow C$

Decomposition

$R_1(A, B)$

$R_2(B, C)$

Now check dependencies:

- $A \rightarrow B$ can be checked in R_1
- $B \rightarrow C$ can be checked in R_2

All dependencies are preserved in the smaller relations.

Therefore, the decomposition is **dependency preserving**.

Example of Non-Dependency Preserving Decomposition

Relation:

$R(A, B, C)$

Functional Dependency:

$A \rightarrow C$

Decomposition

$R_1(A, B)$

$R_2(B, C)$

Now check dependency $A \rightarrow C$:

- R_1 does not contain C
- R_2 does not contain A

Thus the dependency cannot be verified without performing a join.

Hence the decomposition is **not dependency preserving**.

Advantages of Dependency Preserving Decomposition

- Functional dependencies can be checked easily
- No need for join operations to enforce constraints

- Improves database efficiency
- Helps maintain data integrity

Lossless vs Dependency Preserving Decomposition

Lossless Decomposition	Dependency Preserving
Prevent loss of information	Preserve functional dependencies
Join reconstructs original relation	Dependencies can be checked without join
Data reconstruction	Constraint preservation
Essential for correct database design	Important for efficient constraint checking

In database design:

- 3NF decomposition generally ensures both lossless join and dependency preservation.
- BCNF decomposition always ensures a lossless join, but may not preserve all functional dependencies.
- Lossless Decomposition ensures the original table can be reconstructed exactly using joins.
- Lossy Decomposition causes loss of information or the generation of incorrect tuples.
- Dependency Preserving Decomposition ensures that functional dependencies remain enforceable without joining tables.

For good database design, decomposition should ideally be both lossless and dependency preserving.